

Fase 8 — Administración Básica de MariaDB

MGTI. Baltazar Martinez Galla

Continuación directa de la Fase 7. Ambos nodos tienen MariaDB 10.11 instalado, configurado con `utf8mb4`, con `mysql_secure_installation` ejecutado y snapshot `fase07-completa` tomado. En esta fase se realiza la administración inicial del motor: creación del **esquema de prueba del laboratorio** (que se usará en todas las fases de distribución posteriores), gestión de usuarios y privilegios, habilitación del slow query log, respaldo básico con `mysqldump` y conexión remota segura desde el host Windows mediante **DBeaver con túnel SSH**. El acceso a internet (adaptador NAT de la Fase 7) sigue activo en ambos nodos para eventuales actualizaciones. Todo el trabajo se realizará exclusivamente por SSH desde el host, sin abrir la ventana gráfica de VirtualBox.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Navegar las bases de datos del sistema de MariaDB (`information_schema` , `performance_schema` , `mysql` , `sys`) y extraer información administrativa mediante consultas SQL.
2. Diseñar y crear un esquema relacional pensado desde el inicio para su futura distribución (fragmentación horizontal, vertical e híbrida) en fases posteriores.
3. Comprender los cuatro niveles del sistema de privilegios de MariaDB (global, base de datos, tabla y columna) y aplicarlos correctamente con `GRANT` y `REVOKE` .
4. Diferenciar los plugins de autenticación `unix_socket` y `mysql_native_password` , y elegir el adecuado para cada tipo de usuario y contexto.
5. Habilitar y consultar el *slow query log*, interpretando sus entradas para identificar consultas candidatas a optimización.
6. Ejecutar respaldos lógicos con `mysqldump` y comprender sus parámetros esenciales, distinguiendo entre un respaldo de esquema y un respaldo de datos completo.
7. Conectar una herramienta gráfica (DBeaver) al motor de MariaDB de forma segura, usando un **túnel SSH** en lugar de exponer el puerto 3306 directamente a la red.
8. Ejecutar los comandos de monitoreo básico del motor (`SHOW STATUS` , `SHOW PROCESSLIST` , `SHOW ENGINE INNODB STATUS`) e interpretar sus salidas más relevantes.
9. Cerrar la fase con el snapshot `fase08-completa` en ambos nodos, en el estado de mayor preparación alcanzado hasta ahora para las fases de distribución.

B. Conceptos teóricos necesarios

1. Sistema de privilegios de MariaDB: cuatro niveles.

MariaDB controla el acceso mediante una jerarquía de cuatro niveles de granularidad, evaluados de mayor a menor especificidad:

- **Global** (`GRANT ... ON *.*`): el privilegio aplica a cualquier base de datos y cualquier tabla del servidor. Se almacena en la tabla `mysql.user`.
- **Base de datos** (`GRANT ... ON basedatos.*`): aplica a todas las tablas de una base de datos específica. Se almacena en `mysql.db`.
- **Tabla** (`GRANT ... ON basedatos.tabla`): aplica solo a una tabla. Se almacena en `mysql.tables_priv`.
- **Columna** (`GRANT SELECT (col1, col2) ON ...`): aplica a columnas específicas. Se almacena en `mysql.columns_priv`.

Para cada privilegio, MariaDB también evalúa la dirección IP o nombre de host desde el que conecta el usuario (parte `@'host'` en la definición). Un usuario se identifica siempre por el par `usuario@host`, nunca solo por el nombre de usuario.

2. Plugins de autenticación: `unix_socket` y `mysql_native_password`.

MariaDB soporta múltiples *plugins* para verificar la identidad de quien conecta:

- `unix_socket`: el motor pregunta al kernel del sistema operativo si el proceso que intenta conectar pertenece al usuario Unix cuyo nombre coincide con el usuario de MariaDB. Solo funciona desde el mismo servidor (socket local). No hay contraseña que transmitir ni almacenar; la identidad la garantiza el SO. Es el método por defecto para el usuario `root` en Ubuntu.
- `mysql_native_password`: esquema clásico de contraseña con hash SHA1 doble almacenado en `mysql.user`. Funciona tanto desde localhost como desde conexiones TCP remotas. Es el método que utilizará la herramienta DBeaver, los scripts de replicación (Fase 10) y cualquier aplicación que conecte desde otro nodo.

Ambos pueden coexistir en el mismo servidor para distintos usuarios.

3. Las cuatro bases de datos del sistema.

- `information_schema`: base de datos virtual (sin archivos reales) que expone metadatos del servidor — tablas, columnas, índices, vistas, privilegios, etc. Siempre está disponible en modo de solo lectura para cualquier usuario conectado.

- `mysql` : base de datos física que almacena los datos del propio motor — tablas de privilegios (`user` , `db` , `tables_priv`), rutinas almacenadas del sistema, historial de eventos. Solo debe modificarse con instrucciones `GRANT` / `REVOKE` y `CREATE USER` , nunca con `INSERT` / `UPDATE` directos.
- `performance_schema` : base de datos de instrumentación (también virtual) que registra métricas internas del motor — contadores de espera, *memory usage*, ejecución de sentencias. Se leerá en fases de diagnóstico.
- `sys` : conjunto de vistas y procedimientos contruidos sobre `performance_schema` que ofrecen resúmenes legibles para humanos. Disponible desde MariaDB 10.5+.

4. Tipos de logs de MariaDB relevantes para este laboratorio.

- **Error log** (`/var/log/mysql/error.log`): errores de arranque, advertencias del motor y eventos críticos. Siempre activo; es el primer lugar donde buscar problemas.
- **Slow query log**: registra consultas que superan un umbral de tiempo configurable (`long_query_time`). Esencial para detectar consultas mal optimizadas antes de introducir la distribución en fases posteriores. Se habilita en esta fase.
- **General query log**: registra *todas* las consultas recibidas. Tiene un impacto significativo en el rendimiento y en el tamaño del log; solo debe activarse puntualmente para depuración, nunca de forma permanente. **No se habilita en esta fase.**
- **Binary log** (`binlog`): registro de todos los cambios de datos en un formato especial, usado por la replicación. Es la piedra angular de las Fases 10 y 11. Se configurará en detalle entonces; en esta fase solo se introduce el concepto.

5. `mysqldump` : respaldo lógico.

`mysqldump` genera un archivo de texto que contiene las sentencias SQL necesarias para recrear el esquema y/o los datos de una o varias bases de datos. Es el método de respaldo más simple y portable; su principal limitación es que la restauración es lenta en bases de datos grandes (hay que ejecutar miles de `INSERT`) y no es consistente con el estado exacto del binlog (para eso se usa `mariabackup` , que se introduce como concepto en esta fase pero cuyo uso detallado corresponde a la Fase 18 de recuperación). Para los volúmenes de este laboratorio, `mysqldump` es completamente adecuado.

6. Túnel SSH para conexión remota segura.

El parámetro `bind-address = 127.0.0.1` (configurado en la instalación de la Fase 7 y no modificado hasta la Fase 10) significa que MariaDB solo acepta conexiones TCP desde el propio servidor. Una herramienta gráfica en el host Windows no puede conectar directamente al puerto 3306 de una VM. La solución es un **túnel SSH**:

Text

```
DBeaver (Windows) → SSH → bdd-nodo01:22 → reenvío local → 127.0.0.1:3306
```

El cliente SSH abre una conexión cifrada al puerto 22 de `bdd-nodo01` y “reenvía” localmente el tráfico del puerto 3306: desde el punto de vista de MariaDB, la conexión parece provenir de `127.0.0.1` (loopback del propio servidor). Esto mantiene `bind-address` intacto y añade una capa de cifrado al canal de administración, sin abrir ningún puerto adicional en la VM.

7. Diseño del esquema `lab_bdd` : pensado para la distribución.

El esquema que se crea en esta fase no es arbitrario. Cada decisión de diseño anticipa los experimentos de las fases 13 a 16:

- La tabla `clientes` tiene una columna `region ENUM('norte','sur','este','oeste')` : el predicado natural para la **fragmentación horizontal** de la Fase 13.
- La tabla `pedidos` replica ese campo `region` (redundancia controlada) para poder fragmentarse independientemente de `clientes` sin necesidad de joins entre nodos.
- La tabla `productos` mezcla columnas de consulta frecuente (precio, stock) con columnas de detalle pesado (descripcion, ficha_tecnica, imagen_url): la separación natural para la **fragmentación vertical** de la Fase 14.
- Las relaciones entre tablas (claves foráneas) se conservan para trabajar con **consultas distribuidas** en la Fase 16.

C. Procedimiento paso a paso

Paso 1 — Iniciar ambas VMs en modo headless y conectarse por SSH.

Arrancar `bdd-nodo01` y `bdd-nodo02` . La mayor parte del trabajo de esta fase se realiza en `bdd-nodo01` ; los pasos que corresponden a `bdd-nodo02` se indican explícitamente.

Paso 2 — Explorar las bases de datos del sistema.

Antes de crear nada nuevo, familiarizarse con las bases de datos ya existentes ejecutando las consultas de exploración de la sección D.1 dentro de `sudo mariadb` .

Paso 3 — Crear el esquema del laboratorio `lab_bdd` .

Ejecutar el script de creación de tablas de la sección D.2. Este esquema es el mismo en ambos nodos por ahora; en la Fase 9 se diseñará cómo distribuirlo.

Paso 4 — Insertar los datos de prueba.

Ejecutar el script de la sección D.3 para poblar `clientes`, `productos`, `pedidos` y `detalle_pedidos` con un conjunto inicial de datos representativos.

Paso 5 — Crear y verificar los usuarios del laboratorio.

Crear dos usuarios con `mysql_native_password`: `lab_admin@'localhost'` con privilegios amplios sobre `lab_bdd`, y `app_user@'localhost'` con privilegios de solo lectura/escritura de datos (no de estructura). Ver sección D.4.

Paso 6 — Verificar el esquema de privilegios aplicado.

Usar `SHOW GRANTS` e `information_schema` para confirmar que los privilegios quedaron exactamente como se planeó (sección D.5).

Paso 7 — Habilitar el slow query log.

Crear el archivo de configuración `99-lab-logs.cnf` en `mariadb.conf.d/` y reiniciar el servicio para activarlo (sección D.6). Generar una consulta lenta artificial para verificar que se registra.

Paso 8 — Ejecutar los comandos de monitoreo del motor.

Practicar los comandos de diagnóstico instantáneo de MariaDB (`SHOW STATUS`, `SHOW PROCESSLIST`, `SHOW ENGINE INNODB STATUS`) e interpretar las métricas más importantes para el laboratorio (sección D.7).

Paso 9 — Realizar un respaldo básico con `mysqldump`.

Generar respaldos del esquema (solo DDL) y de esquema más datos de `lab_bdd`, guardados en la carpeta de trabajo del laboratorio dentro del nodo (sección D.8).

Paso 10 — Instalar DBeaver en el host Windows y configurar la conexión por túnel SSH.

DBeaver Community Edition es gratuita y no requiere instalación de JDK separado en versiones recientes. La conexión a cada nodo se configura usando las credenciales SSH de `bddadmin` y el usuario `lab_admin` de MariaDB (sección D.9).

Paso 11 — Repetir los Pasos 3 a 9 en `bdd-nodo02`.

El esquema `lab_bdd`, los datos de prueba, los usuarios y la configuración de logs deben ser idénticos en ambos nodos. Esto garantiza que las fases de replicación puedan iniciarse desde un estado coherente.

Paso 12 — Apagar ambas VMs y tomar el snapshot de cierre de fase.

Ver sección D.10.

Paso 13 — Validar con el checklist de la sección G.

D. Comandos completos

Los comandos marcados **(VM)** se ejecutan dentro de una sesión SSH en la VM indicada. Los marcados **(host)** se ejecutan en PowerShell en Windows. Los bloques `MariaDB>` se ejecutan dentro del prompt del motor, accedido con `sudo mariadb`.

D.1 Iniciar las VMs y conectarse por SSH (host)

 PowerShell

```
VBoxManage startvm "bdd-nodo01" --type headless  
VBoxManage startvm "bdd-nodo02" --type headless
```

Esperar 20–30 segundos y conectarse:

 PowerShell

```
ssh bddadmin@192.168.56.101
```

D.2 Explorar las bases de datos del sistema (VM — bdd-nodo01)

 Bash

```
sudo mariadb
```

Dentro del prompt `MariaDB [(none)]>`:

SQL

```
-- Listar todas las bases de datos del sistema
SHOW DATABASES;
-- ¿Qué tablas existen en la base de datos del sistema 'mysql'?
SHOW TABLES IN mysql;
-- Tabla de usuarios: quién tiene acceso y con qué plugin de autenticación
SELECT User, Host, plugin, authentication_string IS NOT NULL
AS tiene_password
FROM mysql.user;
-- Ver todos los privilegios globales actuales
SELECT User, Host, Select_priv, Insert_priv, Super_priv, Repl_slave_priv
FROM mysql.user;
-- Cuántas tablas existen en cada base de datos del sistema
SELECT TABLE_SCHEMA AS base_datos,
       COUNT(*)      AS num_tablas
FROM information_schema.TABLES
GROUP BY TABLE_SCHEMA
ORDER BY num_tablas DESC;
-- Variables de configuración activas más relevantes
SHOW VARIABLES LIKE 'character%';
SHOW VARIABLES LIKE 'collation%';
SHOW VARIABLES LIKE 'bind_address';
SHOW VARIABLES LIKE 'datadir';
SHOW VARIABLES LIKE 'log_error';
EXIT;
```

D.3 Crear el esquema del laboratorio `lab_bdd` (VM — bdd-nodo01)

Crear el archivo del script SQL para mantener el DDL documentado:

```
cat > /tmp/crear_schema_lab_bdd.sql << 'EOF'
-- =====
-- Esquema del Laboratorio de Bases de Datos Distribuidas
-- Creado en la Fase 8. Diseñado para fragmentación futura.
-- =====

CREATE DATABASE IF NOT EXISTS lab_bdd
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE lab_bdd;

-- -----
-- CLIENTES: campo 'region' -> fragmentación horizontal (F13)
-- -----

CREATE TABLE IF NOT EXISTS clientes (
    id            INT            AUTO_INCREMENT PRIMARY KEY,
    nombre        VARCHAR(100) NOT NULL,
    apellido      VARCHAR(100) NOT NULL,
    email         VARCHAR(150) UNIQUE,
    telefono      VARCHAR(20),
    region        ENUM('norte','sur','este','oeste') NOT NULL,
    ciudad        VARCHAR(100),
    fecha_alta    DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB COMMENT='Fragmentación horizontal por region (Fase 13)';

-- -----
-- PRODUCTOS: columnas básicas + detalle -> frag. vertical (F14)
-- -----

CREATE TABLE IF NOT EXISTS productos (
    id            INT            AUTO_INCREMENT PRIMARY KEY,
    sku           VARCHAR(50)    NOT NULL UNIQUE,
    nombre        VARCHAR(150) NOT NULL,
    categoria     VARCHAR(50),
    precio        DECIMAL(10,2) NOT NULL,
    stock         INT           DEFAULT 0,
    -- Columnas de detalle (candidatas a nodo separado en Fase 14)
    descripcion   TEXT,
    ficha_tecnica TEXT,
    imagen_url    VARCHAR(255),
    peso_kg       DECIMAL(6,3),
    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB COMMENT='Fragmentación vertical: basico vs detalle (Fase 14)';

-- -----
-- PEDIDOS: campo 'region' propio -> frag. horizontal (F13)
-- -----

CREATE TABLE IF NOT EXISTS pedidos (
    id            INT            AUTO_INCREMENT PRIMARY KEY,
    cliente_id    INT            NOT NULL,
```



```

region          ENUM('norte','sur','este','oeste') NOT NULL,
fecha_pedido    DATETIME          DEFAULT CURRENT_TIMESTAMP,
estado          ENUM('pendiente','procesado','enviado',
                    'entregado','cancelado') DEFAULT 'pendiente',
total           DECIMAL(10,2),
FOREIGN KEY (cliente_id) REFERENCES clientes(id)
                ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB COMMENT='Fragmentación horizontal por region (Fase 13)';
-- -----
-- DETALLE_PEDIDOS: tabla de relación N:M
-- -----
CREATE TABLE IF NOT EXISTS detalle_pedidos (
    id            INT              AUTO_INCREMENT PRIMARY KEY,
    pedido_id     INT              NOT NULL,
    producto_id   INT              NOT NULL,
    cantidad      INT              NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10,2) NOT NULL,
    subtotal      DECIMAL(10,2) GENERATED ALWAYS AS
                (cantidad * precio_unitario) STORED,
    FOREIGN KEY (pedido_id) REFERENCES pedidos(id)
                ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (producto_id) REFERENCES productos(id)
                ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB COMMENT='Detalle de líneas de pedido';
-- Confirmar la creación
SHOW TABLES;
EOF

```

Ejecutar el script:

 Bash

```
sudo mariadb < /tmp/crear_schema_lab_bdd.sql
```

Verificar que las cuatro tablas existen:

 Bash

```
sudo mariadb -e "SHOW TABLES IN lab_bdd;"
```

D.4 Insertar datos de prueba (VM — bdd-nodo01)

 Bash

```
cat > /tmp/insertar_datos_lab_bdd.sql << 'EOF'
USE lab_bdd;
-- ----- CLIENTES (20 registros, 5 por region) -----
-----
INSERT INTO clientes (nombre, apellido, email, telefono, region,
ciudad) VALUES
-- Norte
('Ana',      'Gutiérrez',  'ana.gutierrez@lab.test',    '8181000001',
'norte',    'Monterrey'),
('Carlos',   'Mendoza',    'carlos.mendoza@lab.test',    '6141000002',
'norte',    'Chihuahua'),
('Laura',    'Ibarra',      'laura.ibarra@lab.test',      '6641000003',
'norte',    'Tijuana'),
('Roberto',  'Soto',        'roberto.soto@lab.test',      '6621000004',
'norte',    'Hermosillo'),
('Verónica', 'Reyes',       'veronica.reyes@lab.test',    '8711000005',
'norte',    'Torreón'),
-- Sur
('Miguel',   'Castro',      'miguel.castro@lab.test',     '9991000006',
'sur',      'Mérida'),
('Sofía',    'Domínguez',  'sofia.dominguez@lab.test',   '9981000007',
'sur',      'Cancún'),
('Jorge',    'Hernández',  'jorge.hernandez@lab.test',   '9511000008',
'sur',      'Oaxaca'),
('Patricia', 'Vázquez',     'patricia.vazquez@lab.test',   '9931000009',
'sur',      'Villahermosa'),
('Ernesto',  'Luna',       'ernesto.luna@lab.test',      '9611000010',
'sur',      'Tuxtla Gtz.'),
-- Este
('Isabel',   'Morales',     'isabel.morales@lab.test',    '2291000011',
'este',     'Veracruz'),
('Héctor',   'Jiménez',   'hector.jimenez@lab.test',    '2281000012',
'este',     'Xalapa'),
('Daniela',  'Torres',     'daniela.torres@lab.test',    '8331000013',
'este',     'Tampico'),
('Alejandro', 'Ríos',       'alejandro.rios@lab.test',    '9211000014',
'este',     'Coatzacoalcos'),
('Fernanda', 'Peña',       'fernanda.pena@lab.test',     '7821000015',
'este',     'Poza Rica'),
-- Oeste
('Arturo',   'Flores',     'arturo.flores@lab.test',     '3331000016',
'oeste',    'Guadalajara'),
('Carmen',   'Rojas',      'carmen.rojas@lab.test',      '3121000017',
'oeste',    'Colima'),
```

```

('Ramón',      'Salinas',      'ramon.salinas@lab.test',      '3111000018',
'oeste',      'Tepic'),
('Lucía',      'Medina',      'lucia.medina@lab.test',      '4431000019',
'oeste',      'Morelia'),
('Eduardo',    'Vargas',      'eduardo.vargas@lab.test',    '4491000020',
'oeste',      'Aguascalientes');
-- ----- PRODUCTOS (10 registros) -----
INSERT INTO productos (sku, nombre, categoria, precio, stock,
                      descripcion, ficha_tecnica, peso_kg) VALUES
('ELEC-001', 'Monitor 24" FHD',      'Electrónica', 3500.00, 25,
'Monitor LED 24 pulgadas Full HD 1920x1080, 75Hz, panel IPS.',
'Resolución: 1920x1080 | Frecuencia: 75 Hz | Panel: IPS | Entradas:
HDMI, VGA',
3.200),
('ELEC-002', 'Teclado Mecánico TKL', 'Electrónica', 950.00, 40,
'Teclado mecánico TKL con switches azules, retroiluminación RGB.',
'Switches: Blue | Layout: TKL 87 teclas | Retroiluminación: RGB |
Conector: USB-C',
0.850),
('ELEC-003', 'Mouse Ergonómico',      'Electrónica', 480.00, 60,
'Mouse inalámbrico ergonómico, sensor óptico 1600 DPI, receptor
USB nano.',
'DPI: 800/1200/1600 | Batería: AA 12 meses | Receptor: nano USB |
Botones: 6',
0.120),
('COMP-001', 'SSD 500 GB SATA',      'Almacenamiento', 1200.00, 30,
'Unidad de estado sólido SATA III 500 GB, velocidad lectura 560 MB/s.',
'Capacidad: 500 GB | Interfaz: SATA III | Lectura: 560 MB/s | Escritura:
520 MB/s',
0.060),
('COMP-002', 'Memoria RAM 16 GB',      'Almacenamiento', 1850.00, 20,
'Módulo de memoria DDR4 16 GB 3200 MHz, latencia CL16.',
'Capacidad: 16 GB | Tipo: DDR4 | Velocidad: 3200 MHz | Latencia: CL16',
0.040),
('RED-001', 'Switch 8 puertos',      'Redes',      650.00, 15,
'Switch no administrado 8 puertos Gigabit Ethernet 10/100/1000.',
'Puertos: 8 x GbE | Capacidad: 16 Gbps | Formato: Sobremesa | PoE: No',
0.350),
('RED-002', 'Cable UTP Cat6 5m',      'Redes',      85.00, 120,
'Cable de red UTP categoría 6 de 5 metros con conectores
RJ45 moldeados.',
'Categoría: Cat6 | Longitud: 5 m | Conector: RJ45 |
Apantallamiento: UTP',
0.100),
('PER-001', 'Silla Gamer Pro',      'Mobiliario', 4200.00, 8,
'Silla de oficina/gaming con soporte lumbar, reposabrazos 4D y
reclinación 135°.',
'Material: Cuero PU | Reclinación: 90-135° | Reposabrazos: 4D | Peso máx:

```

```

120 kg',
18.500),
('PER-002', 'Escritorio L 140 cm', 'Mobiliario', 2800.00, 5,
'Escritorio en forma de L 140x120 cm con superficie de melamina 25 mm.',
'Medidas: 140x120x75 cm | Material: MDP 25 mm | Acabado: Melamina |
Color: Roble',
35.000),
('ACC-001', 'Hub USB-C 7 en 1', 'Accesorios', 420.00, 50,
'Hub multifunción USB-C con HDMI 4K, 3 USB-A 3.0, SD, MicroSD y USB-C
PD 100W.',
'Entradas: 1 USB-C | Salidas: HDMI 4K, 3xUSB-A, SD, MicroSD | PD: 100W',
0.080);
-- ----- PEDIDOS (20 registros, mix de regiones y estados)
-----
INSERT INTO pedidos (cliente_id, region, estado, total) VALUES
( 1, 'norte', 'entregado', 4450.00),
( 2, 'norte', 'enviado', 1430.00),
( 3, 'norte', 'procesado', 3500.00),
( 4, 'norte', 'pendiente', 2800.00),
( 5, 'norte', 'entregado', 735.00),
( 6, 'sur', 'entregado', 5050.00),
( 7, 'sur', 'enviado', 2050.00),
( 8, 'sur', 'procesado', 565.00),
( 9, 'sur', 'pendiente', 4620.00),
(10, 'sur', 'cancelado', 850.00),
(11, 'este', 'entregado', 1200.00),
(12, 'este', 'enviado', 3980.00),
(13, 'este', 'procesado', 6020.00),
(14, 'este', 'pendiente', 505.00),
(15, 'este', 'entregado', 1650.00),
(16, 'oeste', 'entregado', 4650.00),
(17, 'oeste', 'enviado', 3285.00),
(18, 'oeste', 'procesado', 480.00),
(19, 'oeste', 'pendiente', 7000.00),
(20, 'oeste', 'entregado', 3220.00);
-- ----- DETALLE_PEDIDOS -----
INSERT INTO detalle_pedidos (pedido_id, producto_id, cantidad,
precio_unitario) VALUES
(1, 1, 1, 3500.00), (1, 3, 2, 480.00),
(2, 2, 1, 950.00), (2, 3, 1, 480.00),
(3, 1, 1, 3500.00),
(4, 9, 1, 2800.00),
(5, 7, 5, 85.00), (5, 6, 1, 650.00),
(6, 8, 1, 4200.00), (6, 2, 1, 950.00),
(7, 5, 1, 1850.00), (7, 3, 1, 480.00),
(8, 6, 1, 650.00), (8, 7, 1, 85.00),
(9, 4, 1, 1200.00), (9, 5, 1, 1850.00), (9, 8, 1, 4200.00),
(10, 2, 1, 950.00),

```

```

(11, 4, 1, 1200.00),
(12, 1, 1, 3500.00), (12, 10, 1, 420.00),
(13, 8, 1, 4200.00), (13, 9, 1, 2800.00),
(14, 7, 3, 85.00), (14, 6, 1, 650.00),
(15, 5, 1, 1850.00),
(16, 8, 1, 4200.00), (16, 3, 1, 480.00),
(17, 1, 1, 3500.00), (17, 7, 3, 85.00),
(18, 3, 1, 480.00),
(19, 9, 1, 2800.00), (19, 1, 1, 3500.00),
(20, 1, 1, 3500.00), (20, 7, 1, 85.00);
-- Actualizar totales calculados en pedidos
UPDATE pedidos p
SET total = (
    SELECT SUM(subtotal)
    FROM detalle_pedidos dp
    WHERE dp.pedido_id = p.id
);
-- Resumen final
SELECT 'clientes' AS tabla, COUNT(*) AS registros FROM clientes
UNION ALL
SELECT 'productos', COUNT(*) FROM productos
UNION ALL
SELECT 'pedidos', COUNT(*) FROM pedidos
UNION ALL
SELECT 'detalle_pedidos', COUNT(*) FROM
detalle_pedidos;
EOF

```

 Bash

```
sudo mariadb < /tmp/insertar_datos_lab_bdd.sql
```

Verificar el conteo de registros:

 Bash

```
sudo mariadb -e "  
SELECT 'clientes'          AS tabla, COUNT(*) AS registros FROM  
lab_bdd.clientes          UNION ALL  
SELECT 'productos',        COUNT(*)          FROM  
lab_bdd.productos          UNION ALL  
SELECT 'pedidos',          COUNT(*)          FROM  
lab_bdd.pedidos            UNION ALL  
SELECT 'detalle_pedidos',  COUNT(*)          FROM  
lab_bdd.detalle_pedidos;  
"
```

Salida esperada:

 Text

```
+-----+-----+  
| tabla          | registros |  
+-----+-----+  
| clientes       |         20 |  
| productos      |         10 |  
| pedidos        |         20 |  
| detalle_pedidos |         36 |  
+-----+-----+
```

D.5 Crear los usuarios del laboratorio (VM — bdd-nodo01)

 Bash

```
sudo mariadb << 'EOF'

-- -----
-- Usuario: lab_admin
-- Propósito: administración del esquema lab_bdd desde el
--             host Windows vía DBeaver + túnel SSH, y desde
--             scripts de mantenimiento locales.
-- Autenticación: mysql_native_password (necesario para DBeaver)
-- -----
CREATE USER IF NOT EXISTS 'lab_admin'@'localhost'
    IDENTIFIED BY 'LabAdmin_2025!';
GRANT ALL PRIVILEGES ON lab_bdd.* TO 'lab_admin'@'localhost';
-- Acceso a information_schema y performance_schema en modo lectura
-- (para que DBeaver pueda explorar metadatos)
GRANT SELECT ON information_schema.* TO 'lab_admin'@'localhost';
GRANT SELECT ON performance_schema.* TO 'lab_admin'@'localhost';
-- -----
-- Usuario: app_user
-- Propósito: simular un usuario de aplicación; solo puede
--             leer y escribir datos, nunca modificar el esquema.
-- -----
CREATE USER IF NOT EXISTS 'app_user'@'localhost'
    IDENTIFIED BY 'AppUser_2025!';
GRANT SELECT, INSERT, UPDATE, DELETE ON lab_bdd.*
TO 'app_user'@'localhost';
-- -----
-- Aplicar los cambios de privilegios de inmediato
-- -----
FLUSH PRIVILEGES;
-- Verificar que los usuarios quedaron registrados
SELECT User, Host, plugin FROM mysql.user
WHERE User IN ('lab_admin', 'app_user', 'root')
ORDER BY User;
EOF
```

Importante: documentar ambas contraseñas en

`C:\LabBDD\Snapshots-Notas\fase08-notas.txt`. Son credenciales de laboratorio intencionalmente simples; nunca usar contraseñas con este patrón en producción.

D.6 Verificar los privilegios otorgados (VM — bdd-nodo01)

 Bash

```
sudo mariadb << 'EOF'
-- Ver privilegios de cada usuario
SHOW GRANTS FOR 'lab_admin'@'localhost';
SHOW GRANTS FOR 'app_user'@'localhost';
-- Confirmar a través de information_schema
SELECT GRANTEE, TABLE_SCHEMA, PRIVILEGE_TYPE, IS_GRANTABLE
FROM information_schema.SCHEMA_PRIVILEGES
WHERE TABLE_SCHEMA = 'lab_bdd'
ORDER BY GRANTEE, PRIVILEGE_TYPE;
EOF
```

Probar que `app_user` puede leer datos pero NO puede modificar el esquema:

 Bash

```
# Debe funcionar: SELECT es un privilegio de app_user
mariadb -u app_user -p'AppUser_2025!' lab_bdd \
  -e "SELECT COUNT(*) AS total_clientes FROM clientes;"
# Debe fallar con Access denied: DROP no fue otorgado a app_user
mariadb -u app_user -p'AppUser_2025!' lab_bdd \
  -e "DROP TABLE clientes;" 2>&1 | grep -i "access denied\|error"
```

D.7 Habilitar el slow query log (VM — bdd-nodo01 y bdd-nodo02)

Crear el archivo de configuración de logs:

 Bash

```
sudo nano /etc/mysql/mariadb.conf.d/99-lab-logs.cnf
```

Contenido completo (idéntico en ambos nodos):

Text

```
# Configuración de logs para el Laboratorio BDD
# Creado en la Fase 8. Cargado después de 50-server.cnf.
# No editar 50-server.cnf directamente.
[mariadb]
# --- Slow Query Log ---
slow_query_log                = 1
slow_query_log_file           = /var/log/mysql/mariadb-slow.log
long_query_time                = 2
# Registrar también consultas que no usan índices (útil para diagnóstico)
log_queries_not_using_indexes = 0
# --- General Query Log (deshabilitado; activar solo para depuración
# puntual) ---
general_log                   = 0
general_log_file              = /var/log/mysql/mariadb-general.log
```

Guardar con **Ctrl+O**, **Enter**, **Ctrl+X**. Reiniciar MariaDB:

Bash

```
sudo systemctl restart mariadb
sudo systemctl status mariadb
```

Verificar que el slow query log está activo dentro del motor:

Bash

```
sudo mariadb -e "SHOW VARIABLES LIKE 'slow_query%';"
sudo mariadb -e "SHOW VARIABLES LIKE 'long_query_time';"
```

Generar una consulta artificialmente lenta para confirmar que se registra (usa **SLEEP()** para simular una consulta que tarda más de 2 segundos):

Bash

```
sudo mariadb lab_bdd -e "SELECT SLEEP(3), COUNT(*) FROM clientes;"
```

Consultar el log para verificar que la entrada apareció:

 Bash

```
sudo tail -20 /var/log/mysql/mariadb-slow.log
```

Se debe ver una entrada con `Query_time: 3.xxx` apuntando al `SELECT SLEEP(3)` .

D.8 Comandos de monitoreo del motor (VM — bdd-nodo01)

 Bash

```
sudo mariadb << 'EOF'
-- ---- Estado global del servidor ----
-- Conexiones acumuladas desde el último arranque
SHOW GLOBAL STATUS LIKE 'Connections';
-- Conexiones activas en este momento
SHOW GLOBAL STATUS LIKE 'Threads_connected';
-- Consultas ejecutadas desde el arranque
SHOW GLOBAL STATUS LIKE 'Questions';
-- Bytes enviados y recibidos
SHOW GLOBAL STATUS LIKE 'Bytes_%';
-- Operaciones de InnoDB (lecturas/escrituras de páginas)
SHOW GLOBAL STATUS LIKE 'Innodb_pages_%';
-- ---- Procesos activos ----
-- Lista todos los hilos de conexión activos en este momento
SHOW FULL PROCESSLIST;
-- ---- Estado de InnoDB ----
-- Salida extensa; incluye buffer pool, transacciones activas,
-- locks, I/O. Se usa para diagnóstico avanzado.
SHOW ENGINE INNODB STATUS\G
-- ---- Resumen de tablas del laboratorio ----
SELECT TABLE_NAME          AS tabla,
       TABLE_ROWS          AS filas_aprox,
       ROUND(DATA_LENGTH/1024, 1) AS datos_KB,
       ROUND(INDEX_LENGTH/1024, 1) AS indices_KB
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'lab_bdd'
ORDER BY DATA_LENGTH DESC;
-- ---- Consulta de negocio de ejemplo para verificar los datos ----
SELECT c.region,
       COUNT(DISTINCT c.id)   AS clientes,
       COUNT(p.id)           AS pedidos,
       ROUND(SUM(p.total), 2) AS ingresos_total
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.region
ORDER BY ingresos_total DESC;
EOF
```

D.9 Respaldo básico con `mysqldump` (VM — bdd-nodo01)

Crear la carpeta de respaldos dentro del nodo:

 Bash

```
sudo mkdir -p /opt/lab_bdd_backups
sudo chown bddadmin:bddadmin /opt/lab_bdd_backups
```

Respaldo 1 — Solo estructura (DDL):

 Bash

```
mysqldump -u lab_admin -p'LabAdmin_2025!' \
  --no-data \
  --routines \
  --triggers \
  --single-transaction \
  lab_bdd > /opt/lab_bdd_backups/lab_bdd_schema_$(date +%Y%m%d).sql
ls -lh /opt/lab_bdd_backups/
```

Respaldo 2 — Estructura y datos completos:

 Bash

```
mysqldump -u lab_admin -p'LabAdmin_2025!' \
  --routines \
  --triggers \
  --single-transaction \
  --quick \
  lab_bdd > /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
ls -lh /opt/lab_bdd_backups/
wc -l /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
```

Verificar que el archivo de respaldo es válido (primeras y últimas líneas):

 Bash

```
head -20 /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
tail -10 /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
```

La última línea debe ser `-- Dump completed on ...` seguida de la marca de tiempo.

Sobre los parámetros usados:

- `--single-transaction`: toma un snapshot consistente de InnoDB sin bloquear las tablas durante el respaldo (equivalente a un `START TRANSACTION` interno).
- `--quick`: descarga las filas de a una en lugar de cargar toda la tabla en memoria; esencial para bases de datos grandes.
- `--routines` y `--triggers`: incluye procedimientos, funciones y disparadores si los hubiera (en este laboratorio no hay aún, pero es buena práctica incluirlos).

D.10 Conectar DBeaver desde el host Windows con túnel SSH

Instalación de DBeaver (host Windows):

Descargar **DBeaver Community Edition** desde <https://dbeaver.io/download/> (archivo `.exe` o `.zip` para Windows). No requiere instalación de JDK separado en versiones 23+; el JDK embebido viene incluido en el instalador.

Configurar la conexión con túnel SSH en DBeaver:

1. Abrir DBeaver → `Archivo` → `Nueva conexión` → seleccionar **MariaDB**.
2. En la pestaña **Principal**:
 - Host: `127.0.0.1`
 - Puerto: `3306`
 - Base de datos: `lab_bdd`
 - Usuario: `lab_admin`
 - Contraseña: `LabAdmin_2025!`
3. En la pestaña **SSH**:
 - Marcar `Use SSH Tunnel`
 - Host/IP: `192.168.56.101`
 - Puerto SSH: `22`
 - Usuario: `bddadmin`
 - Método de autenticación: `Password`
 - Contraseña SSH: (contraseña definida para `bddadmin` en la Fase 5)
4. Hacer clic en `Test Connection`. DBeaver primero abre el túnel SSH al puerto 22 de la VM y luego conecta al `127.0.0.1:3306` que el túnel reenvía.
5. Si la prueba es exitosa: `Finalizar`.

Repetir los pasos para crear una segunda conexión apuntando a `192.168.56.102` (`bdd-nodo02`), con los mismos usuarios y contraseñas de MariaDB.

Si DBeaver solicita instalar el driver JDBC de MariaDB, aceptar y dejar que lo descargue automáticamente (requiere internet en el host Windows, no en la VM).

D.11 Repetir los Pasos D.3 a D.9 en `bdd-nodo02` (VM — bdd-nodo02)

Conectarse a `bdd-nodo02` por SSH y ejecutar exactamente los mismos scripts.

El contenido de `lab_bdd` debe ser idéntico en ambos nodos:

 PowerShell

```
# Desde el host, abrir una segunda terminal SSH
ssh bddadmin@192.168.56.102
```

Dentro de `bdd-nodo02`, ejecutar en orden:

 Bash

```
# Copiar los scripts desde bdd-nodo01 usando scp (ejecutar desde
bdd-nodo02)
scp bddadmin@192.168.56.101:/tmp/crear_schema_lab_bdd.sql /tmp/
scp bddadmin@192.168.56.101:/tmp/insertar_datos_lab_bdd.sql /tmp/
# Aplicar el esquema y los datos
sudo mariadb < /tmp/crear_schema_lab_bdd.sql
sudo mariadb < /tmp/insertar_datos_lab_bdd.sql
# Crear usuarios
sudo mariadb << 'EOF'
CREATE USER IF NOT EXISTS 'lab_admin'@'localhost'
    IDENTIFIED BY 'LabAdmin_2025!';
GRANT ALL PRIVILEGES ON lab_bdd.* TO 'lab_admin'@'localhost';
GRANT SELECT ON information_schema.* TO 'lab_admin'@'localhost';
GRANT SELECT ON performance_schema.* TO 'lab_admin'@'localhost';
CREATE USER IF NOT EXISTS 'app_user'@'localhost'
    IDENTIFIED BY 'AppUser_2025!';
GRANT SELECT, INSERT, UPDATE, DELETE ON lab_bdd.*
TO 'app_user'@'localhost';
FLUSH PRIVILEGES;
EOF
# Crear configuración de logs (idéntica a bdd-nodo01)
sudo tee /etc/mysql/mariadb.conf.d/99-lab-logs.cnf > /dev/null << 'EOF'
[mariadb]
slow_query_log                = 1
slow_query_log_file            = /var/log/mysql/mariadb-slow.log
long_query_time                = 2
log_queries_not_using_indexes = 0
general_log                   = 0
general_log_file               = /var/log/mysql/mariadb-general.log
EOF
sudo systemctl restart mariadb
# Crear carpeta de respaldos y ejecutar respaldo inicial
sudo mkdir -p /opt/lab_bdd_backups
sudo chown bddadmin:bddadmin /opt/lab_bdd_backups
mysqldump -u lab_admin -p'LabAdmin_2025!' \
    --routines --triggers --single-transaction --quick \
    lab_bdd > /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
# Verificación final
sudo mariadb -e "
    SELECT 'clientes', COUNT(*) FROM lab_bdd.clientes          UNION ALL
    SELECT 'productos', COUNT(*) FROM lab_bdd.productos        UNION ALL
    SELECT 'pedidos', COUNT(*) FROM lab_bdd.pedidos             UNION ALL
    SELECT 'detalle', COUNT(*) FROM lab_bdd.detalle_pedidos;
"
```

D.12 Apagar ambas VMs y tomar el snapshot `fase08-completa` (host)

Desde cada sesión SSH (ejecutar en cada nodo):

```
 Bash  
  
sudo poweroff
```

Confirmar desde el host que ambas VMs están detenidas:

```
 PowerShell  
  
VBoxManage list runningvms
```

La salida debe estar vacía. Tomar los snapshots:

```
 PowerShell  
  
VBoxManage snapshot "bdd-nodo01" take "fase08-completa" \  
--description "lab_bdd creada: 20 clientes, 10 productos, 20 pedidos,  
35 detalles. Usuarios lab_admin y app_user. Slow log activo. Respaldo  
mysqldump generado."  
VBoxManage snapshot "bdd-nodo02" take "fase08-completa" \  
--description "lab_bdd creada: idéntica a bdd-nodo01. Usuarios lab_admin y  
app_user. Slow log activo. Respaldo mysqldump generado."
```

Confirmar los snapshots:

```
 PowerShell  
  
VBoxManage snapshot "bdd-nodo01" list  
VBoxManage snapshot "bdd-nodo02" list
```

Cada VM debe mostrar cuatro snapshots en orden: `fase05-completa` , `fase06-completa` , `fase07-completa` y `fase08-completa` .

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen los siguientes puntos en **ambos nodos** (`bdd-nodo01` y `bdd-nodo02`):

1. `SHOW DATABASES;` dentro de `sudo mariadb` muestra `lab_bdd` además de las cuatro bases de datos del sistema. La base de datos `test` **no** existe.
 2. `SHOW TABLES IN lab_bdd;` muestra exactamente cuatro tablas: `clientes` , `productos` , `pedidos` , `detalle_pedidos` .
 3. La consulta de conteo confirma 20 clientes, 10 productos, 20 pedidos y 36 registros de detalle.
 4. `SHOW GRANTS FOR 'lab_admin'@'localhost';` muestra `GRANT ALL PRIVILEGES ON lab_bdd.*` .
 5. `SHOW GRANTS FOR 'app_user'@'localhost';` muestra únicamente los privilegios `SELECT, INSERT, UPDATE, DELETE` sobre `lab_bdd.*` y ninguno adicional.
 6. El intento de `DROP TABLE` con `app_user` falla con `Access denied` .
 7. `SHOW VARIABLES LIKE 'slow_query_log';` devuelve `ON` .
 8. `SHOW VARIABLES LIKE 'long_query_time';` devuelve `2.000000` .
 9. El archivo `/var/log/mysql/mariadb-slow.log` existe y contiene al menos una entrada del `SELECT SLEEP(3)` ejecutado en la sección D.7.
 10. El archivo de respaldo `lab_bdd_completo_YYYYMMDD.sql` existe en `/opt/lab_bdd_backups/` con un tamaño mayor a cero y la línea `-- Dump completed on` al final.
 11. Desde DBeaver en el host Windows, la conexión con túnel SSH a `bdd-nodo01` y a `bdd-nodo02` abre exitosamente y muestra las tablas de `lab_bdd` .
 12. La conectividad Host-Only sigue intacta: `ping 192.168.56.101` y `ping 192.168.56.102` responden desde el host Windows.
 13. Los snapshots `fase08-completa` existen en `bdd-nodo01` y `bdd-nodo02` .
 14. Las credenciales de `lab_admin` y `app_user` están documentadas en `C:\LabBDD\Snapshots-Notas\fase08-notas.txt` .
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>CREATE USER</code> falla con <code>ERROR 1396: Operation CREATE USER failed</code>	El usuario ya existe de un intento anterior fallido	Usar <code>DROP USER IF EXISTS 'lab_admin'@'localhost';</code> antes de volver a crear, o usar la sintaxis <code>CREATE USER IF NOT EXISTS</code>
<code>SHOW GRANTS</code> devuelve un error de <code>no such grant</code>	El usuario se creó pero <code>GRANT</code> no se ejecutó correctamente, o se especificó un <code>host</code> diferente	Verificar con <code>SELECT User, Host FROM mysql.user;</code> que el usuario y el host coincidan exactamente con lo que se usó en <code>GRANT</code>
<code>mysqldump</code> falla con <code>Access denied; you need the PROCESS privilege</code>	La versión de MariaDB/MySQL es $\geq$ 10.5 y <code>mysqldump</code> necesita <code>SHOW MASTER STATUS</code> al usar <code>--single-transaction</code> sin el flag <code>--no-tablespaces</code>	Añadir el flag <code>--no-tablespaces</code> al comando de <code>mysqldump</code> , o usar <code>root</code> con <code>sudo</code> : <code>sudo mysqldump -u root ...</code>
DBeaver muestra <code>Communications link failure</code> o <code>Connection refused</code>	El túnel SSH no se estableció correctamente antes de intentar la conexión a MariaDB	En DBeaver, verificar que la pestaña SSH tenga la IP del Host-Only (<code>.101 / .102</code>), usuario <code>bddadmin</code> y contraseña correcta; probar primero un cliente SSH externo (PowerShell) para descartar problemas de credenciales SSH
DBeaver muestra <code>Public Key Retrieval is not allowed</code>	El driver JDBC de MariaDB requiere ajuste de parámetros SSL para conexión local	En la configuración de conexión de DBeaver, pestaña <code>Driver properties</code> , añadir la propiedad <code>allowPublicKeyRetrieval = true</code>
El slow query log no registra la consulta <code>SELECT SLEEP(3)</code>	El archivo de configuración <code>99-lab-logs.cnf</code> tiene un error de sintaxis o el servicio no se reinició tras crearlo	Verificar la sintaxis con <code>sudo mariadb -e "SHOW VARIABLES LIKE 'slow_query_log';"</code> ; si devuelve <code>OFF</code> , revisar el archivo de configuración y ejecutar <code>sudo systemctl restart mariadb</code>
<code>scp</code> falla al copiar los scripts de <code>bdd-nodo01</code> a <code>bdd-nodo02</code>	La clave pública SSH no está configurada entre nodos (se usa	Agregar la flag <code>-o StrictHostKeyChecking=no</code> al <code>scp</code> la primera vez e introducir la contraseña cuando se solicite;

	autenticación por contraseña)	alternativamente, copiar el contenido del script manualmente
El conteo de <code>detalle_pedidos</code> da menos de 36	Alguna clave foránea falló durante la inserción, posiblemente porque los <code>producto_id</code> o <code>pedido_id</code> referenciados no existían en ese momento	Verificar el orden de inserción (primero <code>clientes</code> , luego <code>productos</code> , luego <code>pedidos</code> , por último <code>detalle_pedidos</code>); si hay errores de FK, truncar todas las tablas (<code>SET FOREIGN_KEY_CHECKS=0</code>) y reejecutar los scripts en orden
<code>UPDATE pedidos SET total = ...</code> actualiza 0 filas	El subquery de <code>detalle_pedidos</code> no encontró filas porque la inserción anterior falló parcialmente	Verificar con <code>SELECT COUNT(*) FROM detalle_pedidos;</code> si hay filas; si no hay, repetir la inserción de <code>detalle_pedidos</code> antes de ejecutar el <code>UPDATE</code>
<code>systemctl restart mariadb</code> tarda más de 30 segundos o falla	Error de sintaxis en <code>99-lab-logs.cnf</code> que impide que el motor arranque	Revisar el log de arranque: <code>sudo journalctl -u mariadb --no-pager -n 30</code> ; buscar la línea <code>[ERROR]</code> o <code>unknown variable</code> para identificar el parámetro mal escrito

G. Checklist de validación

- ☐ `SHOW DATABASES;` muestra `lab_bdd` en ambos nodos; no existe `test`.
- ☐ Las cuatro tablas (`clientes`, `productos`, `pedidos`, `detalle_pedidos`) existen en `lab_bdd` en ambos nodos con las columnas y tipos correctos.
- ☐ Los datos de prueba están insertados: 20 clientes, 10 productos, 20 pedidos, 36 detalles en **ambos nodos**.
- ☐ `SHOW GRANTS FOR 'lab_admin'@'localhost';` muestra `ALL PRIVILEGES ON lab_bdd.*` en ambos nodos.
- ☐ `SHOW GRANTS FOR 'app_user'@'localhost';` muestra únicamente los privilegios DML (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) sobre `lab_bdd.*` en ambos nodos.
- ☐ El intento de ejecutar `DROP TABLE` con `app_user` devuelve `Access denied`.
- ☐ `/etc/mysql/mariadb.conf.d/99-lab-logs.cnf` existe en ambos nodos con el contenido correcto.
- ☐ `SHOW VARIABLES LIKE 'slow_query_log';` devuelve `ON` en ambos nodos.
- ☐ `/var/log/mysql/mariadb-slow.log` contiene la entrada del `SELECT SLEEP(3)` ejecutado durante la verificación, en ambos nodos.

- ☐ Los archivos de respaldo `mysqldump` existen en `/opt/lab_bdd_backups/` con la línea `-- Dump completed on` al final, en ambos nodos.
- ☐ La conexión DBeaver con túnel SSH funciona hacia `bdd-nodo01` y `bdd-nodo02` y muestra las tablas de `lab_bdd`.
- ☐ La consulta de resumen por región ejecutada en D.8 devuelve 4 filas (norte, sur, este, oeste) con datos de clientes, pedidos e ingresos.
- ☐ La conectividad Host-Only sigue funcionando: `ping 192.168.56.101` y `ping 192.168.56.102` responden desde el host Windows.
- ☐ Ambas VMs se apagaron de forma ordenada.
- ☐ Los snapshots `fase08-completa` existen en `bdd-nodo01` y `bdd-nodo02`.
- ☐ Las credenciales de los usuarios creados están documentadas en `C:\LabBDD\Snapshots-Notas\fase08-notas.txt`.
- ☐ Puedo explicar la diferencia entre los plugins `unix_socket` y `mysql_native_password` y cuándo se usa cada uno.
- ☐ Puedo explicar por qué el esquema `lab_bdd` fue diseñado con el campo `region` y la separación entre columnas básicas y de detalle en `productos`.

Preguntas teóricas para estudiantes

1. MariaDB identifica a los usuarios mediante el par `usuario@host` en lugar de solo por el nombre de usuario. Explica qué implicación tiene esto en el modelo de seguridad: ¿puede existir un usuario `app_user@'localhost'` y al mismo tiempo un `app_user@'192.168.56.%'` con privilegios totalmente distintos? ¿En qué escenario de este laboratorio sería relevante esa diferencia?
2. Describe el flujo completo de una conexión DBeaver al motor MariaDB usando túnel SSH: ¿qué componentes participan, en qué orden establecen la comunicación, y por qué el motor de MariaDB solo ve conexiones provenientes de `127.0.0.1` a pesar de que DBeaver está en una máquina diferente?
3. ¿Qué diferencia hay entre `mysqldump --single-transaction` y un respaldo sin esa opción? ¿Por qué es importante la opción `--single-transaction` para bases de datos InnoDB, y en qué situación **no** sería suficiente para garantizar consistencia?
4. El diseño de la tabla `pedidos` incluye una columna `region` que duplica (de forma controlada) la región del cliente asociado. Discute las ventajas y desventajas de esta redundancia desde la perspectiva de la **fragmentación horizontal** planeada para la Fase 13. ¿Qué problema resuelve esta redundancia al momento de fragmentar?

5. El *slow query log* se configuró con `long_query_time = 2`. Explica qué criterios se usarían para elegir un umbral más bajo (por ejemplo, 0.5 segundos) o más alto (por ejemplo, 10 segundos) en un sistema real, y qué consecuencias tiene sobre el tamaño del log y el rendimiento del servidor cada elección.
-

Ejercicios prácticos

1. **Exploración de metadatos con `information_schema`.**

Escribir una consulta SQL que liste, para cada tabla de `lab_bdd`, el nombre de cada columna, su tipo de dato, si admite NULL, si tiene valor por defecto y si forma parte de una clave primaria o foránea. La consulta debe usar `information_schema.COLUMNS` y `information_schema.KEY_COLUMN_USAGE` con un `JOIN`, y ordenar el resultado por tabla y por posición de columna. Ejecutarla dentro de `sudo mariadb` y documentar los resultados.

2. **Verificación de integridad del respaldo.**

Crear una base de datos temporal `lab_bdd_test` en `bdd-nodo01`, restaurar el respaldo generado con `mysqldump` usando `mariadb lab_bdd_test < archivo.sql`, y ejecutar la misma consulta de conteo de registros en ambas bases de datos (`lab_bdd` y `lab_bdd_test`) para confirmar que los datos son idénticos. Al finalizar, eliminar `lab_bdd_test` con `DROP DATABASE`.

3. **Análisis del slow query log.**

Habilitar temporalmente `log_queries_not_using_indexes = 1` en `99-lab-logs.cnf`, reiniciar MariaDB y ejecutar cinco consultas variadas sobre `lab_bdd` (al menos dos sin cláusula `WHERE` con índice, dos con `JOIN` entre tablas y una con `GROUP BY`). Revisar el slow query log con `sudo tail -50 /var/log/mysql/mariadb-slow.log` e identificar cuáles consultas aparecen y por qué. Restaurar el valor original (`log_queries_not_using_indexes = 0`) al finalizar.

Reto adicional para alumnos avanzados

Implementar un **script de respaldo automatizado** en Bash que: (a) genere un `mysqldump` de `lab_bdd` con fecha y hora en el nombre del archivo, (b) comprima el resultado con `gzip`, © elimine automáticamente los respaldos con más de 7 días de antigüedad para evitar llenar el disco, y (d) registre en un archivo de log propio (`/opt/lab_bdd_backups/backup.log`) la fecha, hora, tamaño del archivo generado y si el proceso fue exitoso o no. El script debe poder

ejecutarse manualmente y también estar preparado para ser invocado por `cron`.
Incluir en la entrega el script comentado y la entrada de `crontab` sugerida para ejecutarlo diariamente a las 02:00 AM.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Creación correcta del esquema <code>lab_bdd</code>	20%	Las cuatro tablas existen en ambos nodos con los tipos de dato correctos, claves foráneas definidas y comentarios que identifican el rol de cada tabla en la distribución futura
Gestión de usuarios y privilegios	20%	<code>lab_admin</code> y <code>app_user</code> existen con exactamente los privilegios descritos; se demuestra que <code>app_user</code> no puede ejecutar DDL
Configuración y verificación de logs	15%	<code>99-lab-logs.cnf</code> existe y está correcto; <code>SHOW VARIABLES</code> confirma el slow log activo; el log muestra al menos una entrada de la consulta de prueba
Respaldo con <code>mysqldump</code>	15%	Se generaron los dos tipos de respaldo (solo DDL y completo); el alumno puede explicar la diferencia entre ambos y el propósito de <code>--single-transaction</code>
Conexión remota con DBeaver y túnel SSH	15%	DBeaver conecta exitosamente a ambos nodos sin errores; el alumno puede explicar por qué se usa un túnel en lugar de abrir directamente el puerto 3306
Comprensión conceptual (preguntas teóricas)	15%	Responde correctamente las preguntas 1 a 5 usando vocabulario técnico apropiado y haciendo referencia al contexto específico del laboratorio

Preparación para la siguiente fase

La **Fase 9: Diseño de la Arquitectura Distribuida** requerirá:

- El esquema `lab_bdd` idéntico en `bdd-nodo01` y `bdd-nodo02` (esta fase).
- Snapshot `fase08-completa` tomado en ambas VMs.

- Comprensión del diseño del esquema: por qué cada tabla tiene los campos que tiene y cómo se mapean a estrategias de fragmentación futuras.
- Acceso SSH y DBeaver funcionando (para consultar y documentar el esquema durante la fase de diseño).

La Fase 9 no instalará ni configurará nada nuevo en las VMs: será una fase teórico-analítica en la que se tomará el esquema `lab_bdd` ya existente y se diseñará, sobre papel y diagramas, el plan de distribución completo que se implementará en las Fases 10 a 16.
